

CHAPTER 1

INTRODUCTION

The **Expense Tracker Web Application** is a full stack web-based system designed to help users efficiently manage, monitor, and analyze their daily financial expenses. In today's fast-paced digital world, individuals often struggle to keep track of their spending habits, which can lead to poor financial planning and unnecessary expenses. This project aims to provide a simple, user-friendly, and reliable platform that allows users to record their income and expenses, categorize transactions, and visualize their financial data clearly.

The Expense Tracker application enables users to log expenses such as food, travel, shopping, bills, and other daily costs in real time. By maintaining a digital record of transactions, the system helps users understand where their money is being spent and encourages better budgeting decisions. The application supports essential operations such as adding new expenses, viewing expense history, updating records, and deleting unwanted entries, ensuring complete control over personal financial data.

The front-end of the application is developed using modern web technologies such as **HTML, CSS, JavaScript, and React.js**, which together provide a responsive and interactive user interface. React.js allows the application to be component-based, improving performance and maintainability while ensuring smooth user interactions. The interface is designed to be simple and intuitive so that even non-technical users can easily navigate the system without confusion.

The back-end of the Expense Tracker is implemented using **Node.js and Express.js**, which handle server-side logic, API requests, and secure communication between the client and the database. The back-end manages user authentication, expense data processing, and CRUD (Create, Read, Update, Delete) operations. RESTful APIs are used to ensure structured and scalable communication between the front end and the server.

For data storage, **MongoDB** is used as the database system due to its flexible, document-oriented structure. MongoDB efficiently stores user details, expense records, categories, and timestamps in a structured yet adaptable format. The use of MongoDB allows the application

to scale easily as the number of users and transactions increases, without requiring complex schema changes.

This Expense Tracker project demonstrates practical implementation of full stack development concepts, including front-end design, back-end API development, database integration, and secure data handling. It reflects real-world application development practices and provides hands-on experience in building a complete web application from scratch. The system can be further enhanced in the future by adding features such as budget limits, graphical expense analytics, monthly reports, and cloud deployment.

Overall, the Expense Tracker Web Application serves as an effective solution for personal finance management while also acting as a strong academic project that showcases the integration of modern web technologies, software engineering principles, and real-world problem-solving skills.

CHAPTER 2

SYSTEM REQUIREMENTS

System requirements describe the essential hardware and software resources needed to develop, deploy, and run the **Expense Tracker Web Application** effectively. Clearly defining system requirements ensures that the application performs smoothly, remains scalable, and provides a reliable user experience. This chapter explains the tools, technologies, and system configurations required for the successful implementation of the project.

2.1 Tools and Technologies

The Expense Tracker application is developed as a **full stack web application**, integrating front-end interfaces, back-end services, and database management systems. Each technology plays a crucial role in ensuring functionality, security, and performance.

Front-End Technologies

The front end of the Expense Tracker is responsible for providing an intuitive and interactive user interface. It allows users to add expenses, view transaction history, analyze spending patterns, and manage budgets easily.

- **HTML5** is used to structure the web pages and define elements such as forms, tables, and dashboards.
- **CSS3** is used for styling the application, ensuring a clean layout, proper alignment, and responsive design across different screen sizes.
- **JavaScript** enables dynamic behavior such as real-time updates, form validation, and interactive components.
- **React.js** is used as the front-end framework to build reusable components, manage application state efficiently, and provide fast rendering for a smooth user experience.

The front-end layer focuses on usability, responsiveness, and ease of navigation, making the application accessible to users with minimal technical knowledge.

Back-End Technologies

The back end of the Expense Tracker handles business logic, user authentication, expense processing, and secure communication between the client and the database.

- **Node.js** is used as the server-side runtime environment, allowing JavaScript to be executed on the server efficiently.

EXPENSE TRACKER

- **Express.js** is used as the back-end framework to manage routing, middleware, and RESTful API development.
- **JWT (JSON Web Token)** is used for secure user authentication and authorization, ensuring that only authenticated users can access their expense data.

The back-end layer ensures data security, validates user requests, and processes CRUD (Create, Read, Update, Delete) operations related to expenses and budgets.

Database Technologies

- **MongoDB** is used as the primary database for storing user information, expense records, categories, and budget details.
- Its document-based and schema-flexible structure makes it suitable for handling dynamic expense data.
- **Mongoose** is used as an Object Data Modeling (ODM) library to define schemas, perform validations, and simplify database operations.

The database layer ensures efficient data storage, retrieval, and scalability as the number of users and transactions increases.

Development Tools

- **Visual Studio Code** is used as the main code editor due to its debugging support, extensions, and integrated terminal.
- **Node Package Manager (NPM)** is used to install, manage, and update project dependencies.
- **Git & GitHub** are used for version control and project management.

Testing and Debugging Tools

- **Postman** is used to test REST API endpoints and verify request-response behavior.
- **Browser Developer Tools** help debug UI issues, inspect network calls, and improve performance.

2.2 Hardware and Software Requirements

Hardware Requirements

The following hardware configuration is sufficient to develop and run the Expense Tracker application smoothly:

- **Processor:** Intel Core i3 or higher (or equivalent)

- **RAM:** Minimum 4 GB (8 GB recommended)
- **Hard Disk:** At least 20 GB free storage
- **Display:** Resolution of 1366×768 or higher
- **Input Devices:** Keyboard and Mouse
- **Internet Connection:** Required for dependency installation, database connectivity, and testing

Software Requirements

The Expense Tracker application requires the following software:

- **Operating System**
 - Windows 10 / 11
 - Linux (Ubuntu or equivalent)
 - macOS
- **Front-End Technologies**
 - HTML5
 - CSS3
 - JavaScript
 - React.js
- **Back-End Technologies**
 - Node.js
 - Express.js
- **Database**
 - MongoDB (Local or MongoDB Atlas)
- **Development Tools**
 - Visual Studio Code
 - Node Package Manager (NPM)

EXPENSE TRACKER

- **Web Browser**
 - Google Chrome
 - Mozilla Firefox
 - Microsoft Edge

CHAPTER 3

IMPLEMENTATION

The implementation phase of the **Expense Tracker Web Application** focuses on transforming the system design into a fully functional application. This phase involves setting up the development environment, configuring the backend and frontend modules, integrating the database, and ensuring smooth interaction between all components. The implementation follows a modular full-stack architecture, making the application scalable, maintainable, and easy to enhance.

The Expense Tracker system is developed using the **MERN stack**, where the frontend handles user interaction, the backend manages business logic and API requests, and the database stores persistent expense-related data. Proper separation of concerns is maintained to ensure clarity and efficiency in development.

3.1 Steps to Execute

Step 1: Install Required Software

To begin the implementation, all necessary software tools must be installed on the system. This includes **Node.js**, which provides the runtime environment for executing JavaScript on the server side and also installs **Node Package Manager (NPM)** for dependency management. **MongoDB** is installed either locally or accessed via MongoDB Atlas for database operations. A code editor such as **Visual Studio Code** is used for writing and managing the project source code. A modern web browser like **Google Chrome** is required for testing and debugging the application.

Step 2: Set Up the Project Directory

A main project folder is created to organize the application files. Inside this folder, two separate directories are maintained: one for the **frontend** and one for the **backend**. This structure helps in managing the project efficiently and supports independent development of client-side and server-side components. The entire project directory is opened in Visual Studio Code for development.

Step 3: Backend Setup

The backend directory is initialized as a Node.js project using NPM. Required backend dependencies such as **Express.js**, **Mongoose**, **CORS**, **dotenv**, and **JSON Web Token (JWT)** are installed. Express.js is used to create the server and define RESTful API routes, while middleware functions handle request parsing, authentication, and error handling. Environment variables are configured to store sensitive information like database connection strings and secret keys securely.

Step 4: Database Configuration

MongoDB is configured as the primary database for storing user details, expense records, and category information. Using **Mongoose**, schemas and models are created to define the structure of expense data, including fields such as amount, category, date, description, and user reference. The database connection is established between the backend server and MongoDB, ensuring reliable data storage and retrieval.

Step 5: Start Backend Server

Once the backend configuration is complete, the server is started using Node.js. Successful startup confirms that the server is running and connected to the database. API endpoints such as adding expenses, retrieving expense lists, updating records, and deleting expenses are tested using tools like Postman to verify proper request handling and responses.

Step 6: Frontend Setup

The frontend directory is initialized using a React application setup. Required frontend dependencies are installed to support routing, state management, and API communication. React components are created for features such as user login, dashboard display, expense entry forms, expense list views, and summary sections. API calls are implemented to fetch and send data between the frontend and backend.

Step 7: Start Frontend Application

The React development server is started, and the application loads in the web browser. The frontend interface is tested to ensure that all components render correctly and interact smoothly with backend APIs. User actions such as adding an expense or viewing expense history are verified at this stage.

Step 8: User Authentication and Access

User authentication functionality is implemented to ensure secure access to the application. Users can register using valid credentials and log in to access their personal expense dashboard. Authentication tokens are used to validate user sessions and restrict unauthorized access to protected routes.

Step 9: Expense Management Operations

Users can add new expenses by entering details such as amount, category, date, and description. The application allows users to view all recorded expenses, edit existing entries, and delete unwanted records. All operations are performed in real time, and changes are reflected immediately in the database and user interface.

3.2 Code Explanation

The Expense Tracker application consists of multiple modules that work together to provide complete functionality. The frontend code manages user interactions and displays data dynamically, while the backend code handles request processing, authentication, and database operations. The database models ensure structured storage of expense data. This modular coding approach improves readability, reusability, and maintainability of the system.

App.js

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Login from "../pages/Login";
import Register from "../pages/Register";
import Dashboard from "../pages/Dashboard";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Api.js

```
import axios from "axios";
const api = axios.create({
  baseURL: "http://localhost:5001/api",
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token && token !== 'undefined' && token !== 'null') config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

```
});
```

```
api.interceptors.response.use(  
  (response) => response,  
  (error) => {  
    if (error.response?.status === 401) {  
      localStorage.removeItem("token");  
      window.location.href = "/login";  
    }  
    return Promise.reject(error);  
  }  
);
```

```
export default api;
```

Budgetmanager.js

```
import axios from "axios";  
const api = axios.create({  
  baseURL: "http://localhost:5001/api",  
});
```

```
api.interceptors.request.use((config) => {  
  const token = localStorage.getItem("token");  
  if (token && token !== 'undefined' && token !== 'null') config.headers.Authorization =  
  `Bearer ${token}`;  
  return config;  
});
```

```
api.interceptors.response.use(  
  (response) => response,  
  (error) => {  
    if (error.response?.status === 401) {  
      localStorage.removeItem("token");  
      window.location.href = "/login";  
    }  
  }  
);
```

```
    }  
    return Promise.reject(error);  
  }  
);
```

export default api;

ExpenseForm.js

```
import { useState } from "react";  
import api from "../api";
```

```
const CATEGORIES = [  
  "Food & Dining",  
  "Transportation",  
  "Shopping",  
  "Bills & Utilities",  
  "Entertainment",  
  "Healthcare",  
  "Education",  
  "Travel",  
  "Personal Care",  
  "Other"  
];
```

```
export default function ExpenseForm({ refresh, editingExpense, onCancel }) {  
  const [amount, setAmount] = useState(editingExpense?.amount || "");  
  const [category, setCategory] = useState(editingExpense?.category || "");  
  const [note, setNote] = useState(editingExpense?.note || "");  
  const [date, setDate] = useState(  
    editingExpense?.date ? new Date(editingExpense.date).toISOString().split('T')[0] : new  
    Date().toISOString().split('T')[0]  
  );  
  const [loading, setLoading] = useState(false);
```

```
const submit = async (e) => {
  e?.preventDefault();
  if (!amount || !category) {
    alert("Please fill in amount and category");
    return;
  }

  setLoading(true);
  try {
    const data = {
      amount: parseFloat(amount),
      category,
      note,
      date: new Date(date)
    };

    if (editingExpense) {
      await api.put(`/expenses/${editingExpense._id}`, data);
    } else {
      await api.post("/expenses", data);
    }

    setAmount("");
    setCategory("");
    setNote("");
    setDate(new Date().toISOString().split('T')[0]);
    refresh();
    if (onCancel) onCancel();
  } catch (err) {
    alert(err.response?.data?.error || "Error saving expense");
  } finally {
    setLoading(false);
  }
}
```

```
};

return (
  <form onSubmit={submit} style={styles.form}>
    <h3 style={styles.title}>{editingExpense ? "Edit Expense" : "Add New Expense"}</h3>
    <div style={styles.inputGroup}>
      <label style={styles.label}>Amount (₹)</label>
      <input
        type="number"
        step="0.01"
        placeholder="0.00"
        value={amount}
        onChange={e => setAmount(e.target.value)}
        style={styles.input}
        required
      />
    </div>
    <div style={styles.inputGroup}>
      <label style={styles.label}>Category</label>
      <select
        value={category}
        onChange={e => setCategory(e.target.value)}
        style={styles.input}
        required
      >
        <option value="">Select Category</option>
        {CATEGORIES.map(cat => (
          <option key={cat} value={cat}>{cat}</option>
        ))}
      </select>
    </div>
    <div style={styles.inputGroup}>
      <label style={styles.label}>Date</label>
```

```
<input
  type="date"
  value={date}
  onChange={e => setDate(e.target.value)}
  style={styles.input}
  required
/>
</div>
<div style={styles.inputGroup}>
  <label style={styles.label}>Note</label>
  <input
    placeholder="Optional note"
    value={note}
    onChange={e => setNote(e.target.value)}
    style={styles.input}
  />
</div>
<div style={styles.buttonGroup}>
  <button type="submit" style={styles.submitButton} disabled={loading}>
    {loading ? "Saving..." : editingExpense ? "Update" : "Add Expense"}
  </button>
  {editingExpense && (
    <button type="button" onClick={onCancel} style={styles.cancelButton}>
      Cancel
    </button>
  )}
</div>
</form>
);
}
```



```
const styles = {
  form: {
```

```
    backgroundColor: "#fff",
    padding: "24px",
    borderRadius: "12px",
    boxShadow: "0 2px 8px rgba(0,0,0,0.1)",
    marginBottom: "24px"
  },
  title: {
    margin: "0 0 20px 0",
    color: "#333",
    fontSize: "20px",
    fontWeight: "600"
  },
  inputGroup: {
    marginBottom: "16px"
  },
  label: {
    display: "block",
    marginBottom: "6px",
    color: "#555",
    fontSize: "14px",
    fontWeight: "500"
  },
  input: {
    width: "100%",
    padding: "10px 12px",
    border: "1px solid #ddd",
    borderRadius: "6px",
    fontSize: "14px",
    boxSizing: "border-box"
  },
  buttonGroup: {
    display: "flex",
    gap: "10px"
```

```
    },
    submitButton: {
      padding: "10px 24px",
      backgroundColor: "#4CAF50",
      color: "white",
      border: "none",
      borderRadius: "6px",
      fontSize: "14px",
      fontWeight: "600",
      cursor: "pointer",
      flex: 1
    },
    cancelButton: {
      padding: "10px 24px",
      backgroundColor: "#f44336",
      color: "white",
      border: "none",
      borderRadius: "6px",
      fontSize: "14px",
      fontWeight: "600",
      cursor: "pointer"
    }
  }
};
```

FinancialReport.js

```
import { useState, useEffect } from "react";
import { Bar, Doughnut, Line } from "react-chartjs-2";
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  ArcElement,
  PointElement,
```

```
LineElement,  
Title,  
Tooltip,  
Legend  
} from "chart.js";  
import api from "../api";
```

```
ChartJS.register(  
  CategoryScale,  
  LinearScale,  
  BarElement,  
  ArcElement,  
  PointElement,  
  LineElement,  
  Title,  
  Tooltip,  
  Legend  
);
```

```
export default function FinancialReports() {  
  const [analytics, setAnalytics] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [selectedMonth, setSelectedMonth] = useState(new Date().getMonth() + 1);  
  const [selectedYear, setSelectedYear] = useState(new Date().getFullYear());  
  
  const loadAnalytics = async () => {  
    setLoading(true);  
    try {  
      const res = await api.get("/expenses/analytics", {  
        params: { month: selectedMonth, year: selectedYear }  
      });  
      setAnalytics(res.data);  
    } catch (err) {
```

```
    console.error("Error loading analytics:", err);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  loadAnalytics();
}, [selectedMonth, selectedYear]);

if (loading) {
  return <div style={styles.loading}>Loading reports...</div>;
}

if (!analytics) {
  return <div style={styles.empty}>No data available</div>;
}

const categoryData = {
  labels: analytics.byCategory.map(item => item._id),
  datasets: [
    {
      label: "Expenses by Category",
      data: analytics.byCategory.map(item => item.total),
      backgroundColor: [
        "#FF6384",
        "#36A2EB",
        "#FFCE56",
        "#4BC0C0",
        "#9966FF",
        "#FF9F40",
        "#FF6384",
        "#C9CBCF",
      ],
    },
  ],
};
```

```
        "#4BC0C0",
        "#FF6384"
    ]
}
]
};

const monthlyData = {
  labels: analytics.monthlyExpenses
    .reverse()
    .map(item => `${item._id.month}/${item._id.year}`),
  datasets: [
    {
      label: "Monthly Expenses (₹)",
      data: analytics.monthlyExpenses.map(item => item.total),
      borderColor: "#2196F3",
      backgroundColor: "rgba(33, 150, 243, 0.1)",
      tension: 0.4
    }
  ]
};

const categoryBarData = {
  labels: analytics.byCategory.map(item => item._id),
  datasets: [
    {
      label: "Amount (₹)",
      data: analytics.byCategory.map(item => item.total),
      backgroundColor: "#4CAF50"
    }
  ]
};
```

```
const currentMonth = new Date().toLocaleString('default', { month: 'long' });
const months = Array.from({ length: 12 }, (_, i) => i + 1);
const years = Array.from({ length: 5 }, (_, i) => new Date().getFullYear() - i);
```

```
const exportToCSV = () => {
  const csvRows = [];
  csvRows.push("Category,Total Amount,Transactions");
  analytics.byCategory.forEach(item => {
    csvRows.push(`${item._id},${item.total.toFixed(2)},${item.count}`);
  });
  csvRows.push(`\nTotal Expenses,${analytics.totalExpenses.toFixed(2)},`);
}
```

```
const csvContent = csvRows.join("\n");
const blob = new Blob([csvContent], { type: "text/csv" });
const url = window.URL.createObjectURL(blob);
const a = document.createElement("a");
a.href = url;
a.download = `expense-report-${selectedMonth}-${selectedYear}.csv`;
a.click();
window.URL.revokeObjectURL(url);
};
```

```
return (
  <div style={styles.container}>
    <div style={styles.header}>
      <h2 style={styles.title}>Financial Reports</h2>
      <div style={styles.headerRight}>
        <div style={styles.filters}>
          <select
            value={selectedMonth}
            onChange={e => setSelectedMonth(parseInt(e.target.value))}
            style={styles.select}
          >
```

```
{months.map(month => (  
  <option key={month} value={month}>  
    {new Date(2000, month - 1).toLocaleString('default', { month: 'long' })}  
  </option>  
))}  
</select>  
  
<select  
  value={selectedYear}  
  onChange={e => setSelectedYear(parseInt(e.target.value))}  
  style={styles.select}  
>  
  {years.map(year => (  
    <option key={year} value={year}>{year}</option>  
  ))}  
</select>  
  
<button onClick={exportToCSV} style={styles.exportButton}>  
  📄 Export CSV  
</button>  
</div>  
</div>  
</div>  
  
<div style={styles.summary}>  
  <div style={styles.summaryCard}>  
    <div style={styles.summaryLabel}>Total Expenses</div>  
    <div style={styles.summaryValue}>  
      ₹ {analytics.totalExpenses.toFixed(2)}  
    </div>  
  </div>  
  <div style={styles.summaryCard}>  
    <div style={styles.summaryLabel}>Categories</div>  
    <div style={styles.summaryValue}>{analytics.byCategory.length}</div>  
  </div>  
</div>
```

```
<div style={styles.summaryCard}>
  <div style={styles.summaryLabel}>Top Category</div>
  <div style={styles.summaryValue}>
    {analytics.byCategory[0]?._id || "N/A"}
  </div>
</div>
</div>

<div style={styles.chartsGrid}>
  <div style={styles.chartCard}>
    <h3 style={styles.chartTitle}>Expenses by Category</h3>
    <div style={styles.chartContainer}>
      <Doughnut
        data={categoryData}
        options={{
          responsive: true,
          maintainAspectRatio: true,
          plugins: {
            legend: {
              position: "bottom"
            }
          }
        }}
      />
    </div>
  </div>

  <div style={styles.chartCard}>
    <h3 style={styles.chartTitle}>Category Breakdown</h3>
    <div style={styles.chartContainer}>
      <Bar
        data={categoryBarData}
        options={{
```

```
        responsive: true,
        maintainAspectRatio: true,
        plugins: {
            legend: {
                display: false
            }
        },
        scales: {
            y: {
                beginAtZero: true
            }
        }
    }}
/>
</div>
</div>
</div>

<div style={styles.chartCard}>
    <h3 style={styles.chartTitle}>Monthly Trend (Last 6 Months)</h3>
    <div style={styles.chartContainer}>
        <Line
            data={monthlyData}
            options={{
                responsive: true,
                maintainAspectRatio: true,
                plugins: {
                    legend: {
                        display: false
                    }
                },
                scales: {
                    y: {
```

```
        beginAtZero: true
      }
    }
  }}
/>
</div>
</div>

<div style={styles.tableCard}>
  <h3 style={styles.chartTitle}>Category Details</h3>
  <table style={styles.table}>
    <thead>
      <tr>
        <th style={styles.th}>Category</th>
        <th style={styles.th}>Total Amount</th>
        <th style={styles.th}>Transactions</th>
      </tr>
    </thead>
    <tbody>
      {analytics.byCategory.map((item, index) => (
        <tr key={index}>
          <td style={styles.td}>{item._id}</td>
          <td style={styles.td}>₹ {item.total.toFixed(2)}</td>
          <td style={styles.td}>{item.count}</td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
</div>
);
}
```

```
const styles = {
  container: {
    padding: "24px"
  },
  header: {
    display: "flex",
    justifyContent: "space-between",
    alignItems: "center",
    marginBottom: "24px"
  },
  title: {
    margin: 0,
    color: "#333",
    fontSize: "24px",
    fontWeight: "600"
  },
  headerRight: {
    display: "flex",
    alignItems: "center",
    gap: "12px"
  },
  filters: {
    display: "flex",
    gap: "12px"
  },
  exportButton: {
    padding: "8px 16px",
    backgroundColor: "#4CAF50",
    color: "white",
    border: "none",
    borderRadius: "6px",
    fontSize: "14px",
    fontWeight: "600",
  },
}
```

```
    cursor: "pointer",
    transition: "background-color 0.2s"
  },
  select: {
    padding: "8px 12px",
    border: "1px solid #ddd",
    borderRadius: "6px",
    fontSize: "14px"
  },
  summary: {
    display: "grid",
    gridTemplateColumns: "repeat(auto-fit, minmax(200px, 1fr))",
    gap: "16px",
    marginBottom: "24px"
  },
  summaryCard: {
    backgroundColor: "#fff",
    padding: "20px",
    borderRadius: "12px",
    boxShadow: "0 2px 8px rgba(0,0,0,0.1)",
    textAlign: "center"
  },
  summaryLabel: {
    fontSize: "14px",
    color: "#666",
    marginBottom: "8px"
  },
  summaryValue: {
    fontSize: "28px",
    fontWeight: "700",
    color: "#333"
  },
  chartsGrid: {
```

```
display: "grid",
gridTemplateColumns: "repeat(auto-fit, minmax(400px, 1fr))",
gap: "24px",
marginBottom: "24px"
},
chartCard: {
  backgroundColor: "#fff",
  padding: "24px",
  borderRadius: "12px",
  boxShadow: "0 2px 8px rgba(0,0,0,0.1)",
  marginBottom: "24px"
},
chartTitle: {
  margin: "0 0 20px 0",
  color: "#333",
  fontSize: "18px",
  fontWeight: "600"
},
chartContainer: {
  height: "300px",
  position: "relative"
},
tableCard: {
  backgroundColor: "#fff",
  padding: "24px",
  borderRadius: "12px",
  boxShadow: "0 2px 8px rgba(0,0,0,0.1)"
},
table: {
  width: "100%",
  borderCollapse: "collapse"
},
th: {
```

```
padding: "12px",
textAlign: "left",
borderBottom: "2px solid #e0e0e0",
color: "#333",
fontWeight: "600"
},
td: {
padding: "12px",
borderBottom: "1px solid #e0e0e0",
color: "#666"
},
loading: {
textAlign: "center",
padding: "40px",
color: "#999"
},
empty: {
textAlign: "center",
padding: "40px",
color: "#999"
}
};
```

Server.js

```
import express from "express";
import mongoose from "mongoose";
import cors from "cors";
import { MONGO_URI } from "./config.js";

import authRoutes from "./routes/authRoutes.js";
import expenseRoutes from "./routes/expenseRoutes.js";
import budgetRoutes from "./routes/budgetRoutes.js";

const app = express();
```

```
app.use(cors());
app.use(express.json());

mongoose.connect(MONGO_URI)
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log("DB Error:", err));

app.use("/api/auth", authRoutes);
app.use("/api/expenses", expenseRoutes);
app.use("/api/budgets", budgetRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

3.3 Screen Shots

Fig 3.3.1 Login Page

This page allows users to log in to the Expense Tracker system using their registered credentials. It ensures secure access to personal expense data and prevents unauthorized usage of the application.

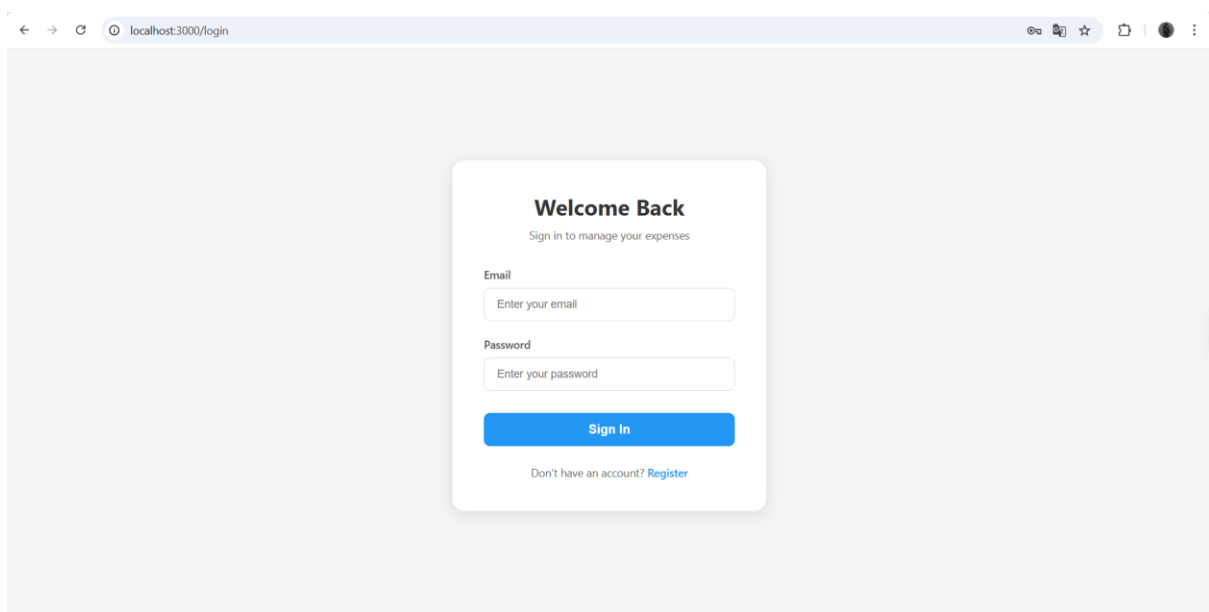


Fig 3.3.2 Expense Dashboard Page

The dashboard page displays an overview of the user's expenses, including total expenditure and recent transactions. It serves as the central control panel for managing expenses efficiently.

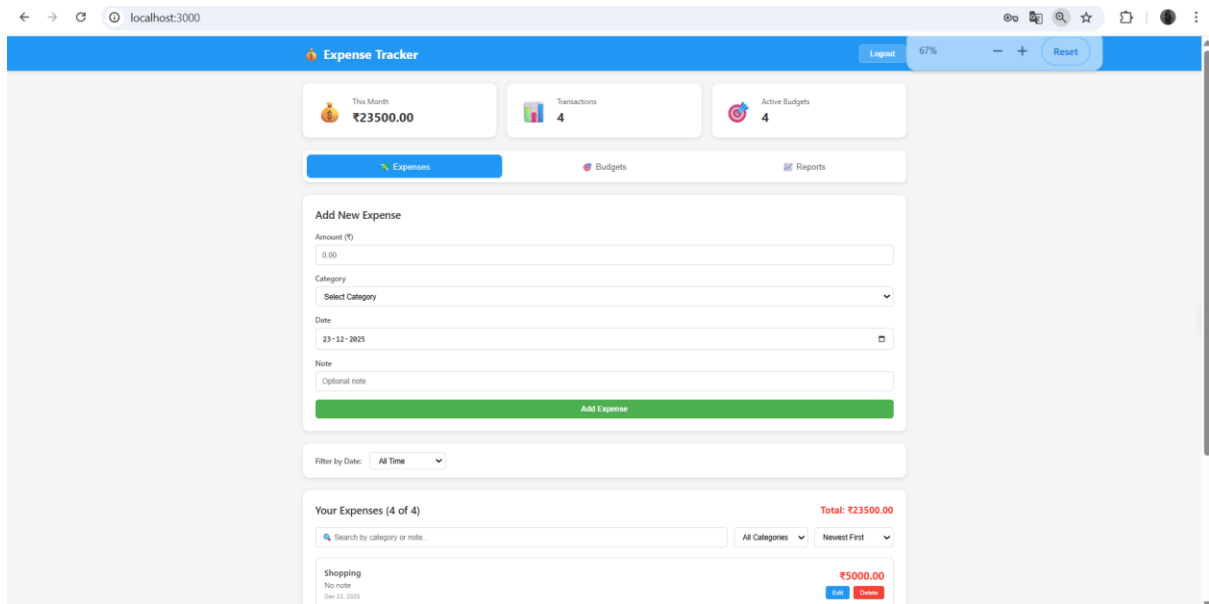


Fig 3.3.3 Add Budget Page

This page enables users to add a new budget by entering required details such as amount, category, date, and description. The form includes validation to ensure accurate data entry.

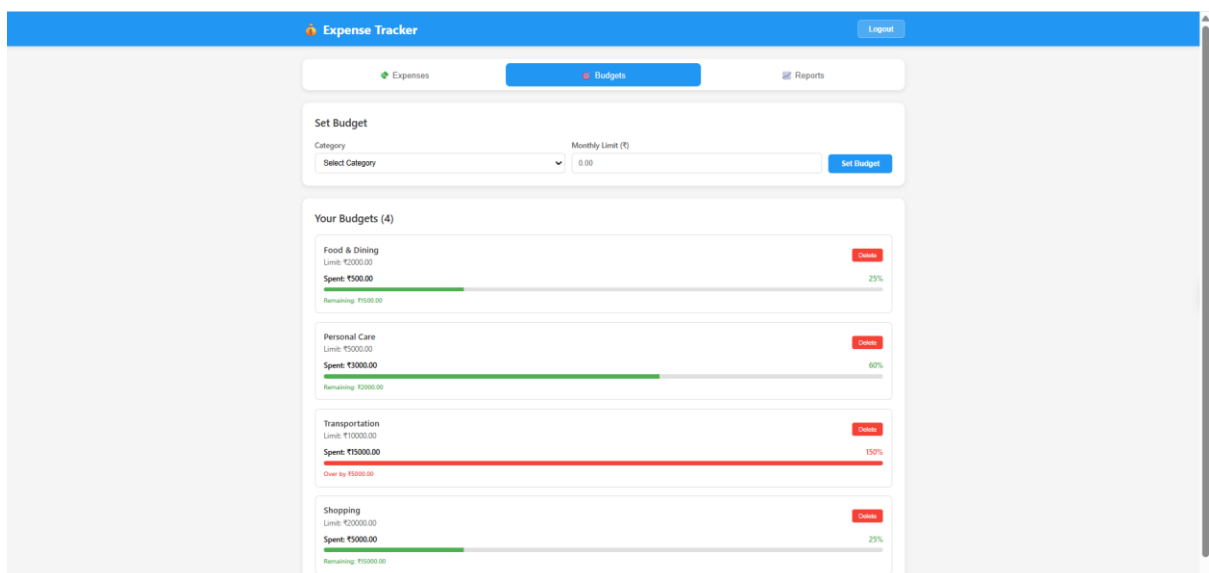


Fig 3.3.4 Reports

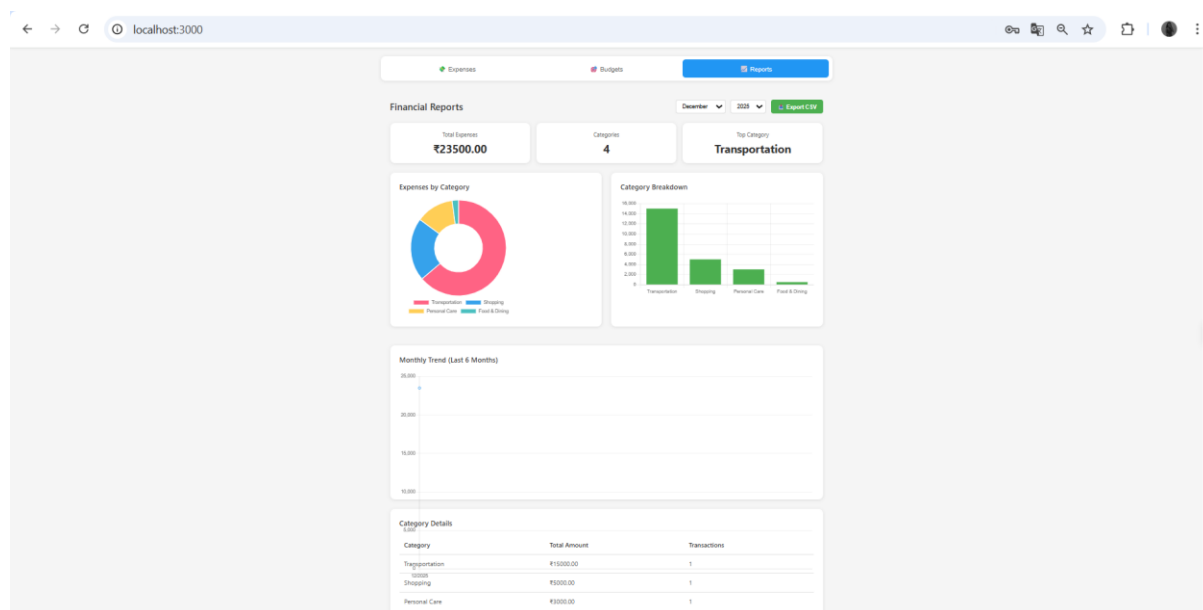
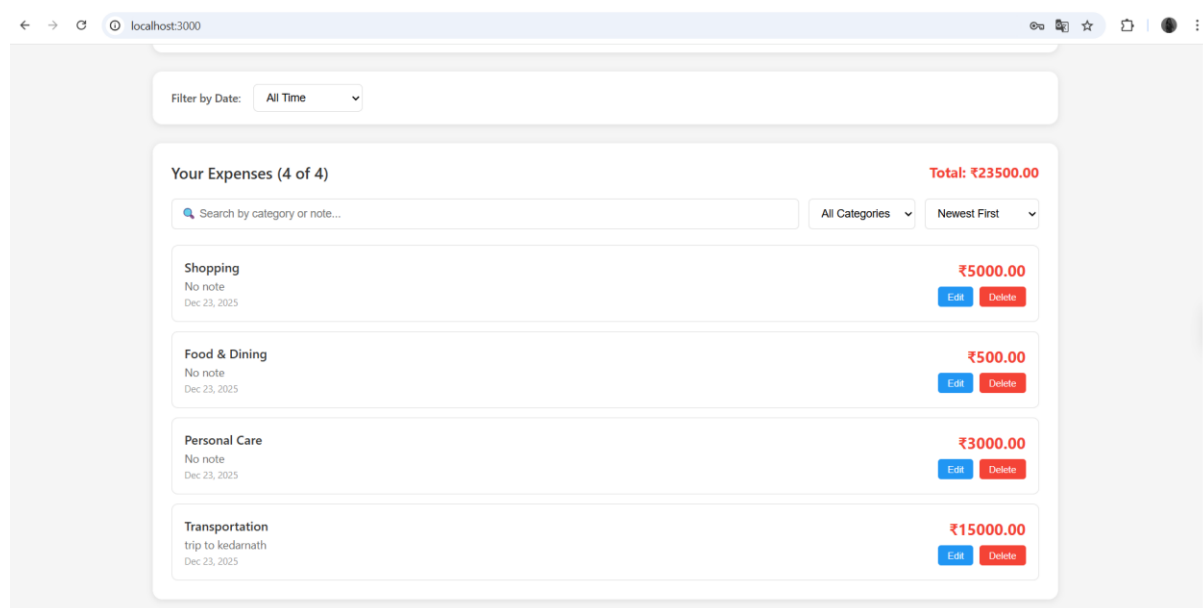


Fig 3.3.5 Expense List and Database View

This section displays the stored expense records retrieved from the database. It allows users to view, update, or delete expenses. From a development perspective, this also confirms that the backend and database integration is functioning correctly.



EXPENSE TRACKER

MongoDB Compass - 127.0.0.1:27018/expense_tracker.users

Connections Edit View Collection Help

budgets users +

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

localhost:27017

- admin
- campus_connect
- config
- local

127.0.0.1:27018

- admin
- config
- expense_tracker
 - budgets
 - expenses
 - users
- local

127.0.0.1:27018 > expense_tracker > users

Documents 2 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

25 1 - 4 of 4

```
{ "_id": ObjectId("693a551e44ce7ed2776388e"),  
  "name": "Rakshitha P",  
  "email": "rakshithap107@gmail",  
  "password": "$2a$10$.LQ8FkGywoKDbZaUPk7NXeUuVWNgI8XrAmkL3eE.fz025GKXg8FDa",  
  "__v": 0 }
```

```
{ "_id": ObjectId("694232e1ee24a284f9606c0"),  
  "name": "Rakshitha P",  
  "email": "rakshithap1234@gmail",  
  "password": "$2a$10$.rWp9fWq4SS6UeXWudE3a6na1B3J3RnA7FghJQ3Ue7gr3IXRQNGF5S.",  
  "__v": 0 }
```

```
{ "_id": ObjectId("694a82b7b815a669566d611b"),  
  "name": "Rakshitha P",  
  "email": "rakshithap1234@gmail",  
  "password": "$2a$10$.rWp9fWq4SS6UeXWudE3a6na1B3J3RnA7FghJQ3Ue7gr3IXRQNGF5S.",  
  "__v": 0 }
```

```
{ "_id": ObjectId("694a82b7b815a669566d611b"),  
  "name": "Harshitha",  
  "email": "harshitha11@gmail",  
  "password": "$2a$10$.p6mRgHdpYdpwnKKA58h0i.uNHYPkuCwaNYMc97rEM4HEWJ39q8Loa",  
  "__v": 0 }
```

MongoDB Compass - 127.0.0.1:27018/expense_tracker

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

localhost:27017

- admin
- campus_connect
- config
- local

127.0.0.1:27018

- admin
- config
- expense_tracker
 - budgets
 - expenses
 - users
- local

expense_tracker +

127.0.0.1:27018 > expense_tracker

Open MongoDB shell Create collection Refresh

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
budgets	-	36.86 kB	5	104.00 B	1	36.86 kB
expenses	-	32.77 kB	8	130.00 B	1	32.77 kB
users	-	36.86 kB	2	152.00 B	2	73.73 kB

CHAPTER 4

CONCLUSION

The **Expense Tracker Web Application** has been successfully designed and implemented as a full stack web project, fulfilling the primary objective of helping users manage and monitor their personal expenses efficiently. The system provides a simple, user-friendly, and reliable platform where users can record daily expenses, categorize them, analyze spending patterns, and maintain better financial discipline. By integrating front-end, back-end, and database components, the project demonstrates a complete real-world full stack development workflow.

The application uses **modern web technologies** to ensure smooth functionality and responsiveness. The front end offers an interactive interface that allows users to add, edit, delete, and view expenses easily. Clear form validations, intuitive layouts, and organized dashboards improve the overall user experience. The back end handles business logic, secure data processing, and API communication, ensuring that all operations such as expense storage, retrieval, and updates are performed accurately. The database layer securely stores user and expense data, making the system scalable and reliable.

One of the major strengths of this project is its **practical usefulness**. Expense tracking is a real-life requirement for students, working professionals, and households. This system enables users to gain insights into their spending habits through categorized expenses and summaries, helping them make informed financial decisions. The project also provides hands-on exposure to important concepts such as CRUD operations, RESTful APIs, database connectivity, authentication, and modular coding practices.

From a learning perspective, this project has significantly improved understanding of **full stack development concepts**, including client-server architecture, database integration, and real-time data handling. It demonstrates how individual components work together to form a complete application. The project also highlights the importance of proper planning, requirement analysis, and systematic implementation in software development.

In conclusion, the **Expense Tracker Web Application** is a successful full stack project that meets all functional requirements and demonstrates efficient use of modern technologies. It serves as a strong foundation for future enhancements such as budget planning, graphical expense analytics, monthly reports, data export features, and cloud deployment. This project effectively proves the applicability of full stack development in solving real-world problems and provides valuable practical experience in building scalable web applications.